

9.1 El Uso de `this` y Listas de Inicialización (*Initializer Lists*)

La palabra reservada `this` hace referencia a la instancia actual de la clase. En Dart, su uso explícito (`this.propiedad`) se reserva principalmente para dos escenarios:

1. **Initializing Formal Parameters:** Asignación directa en la firma del constructor (`Clase(this.propiedad);`).
2. **Desambiguación:** Evitar colisiones de nombres entre parámetros y variables de instancia cuando tienen el mismo identificador.

Regla de estilo Effective Dart: Si no hay sombra de nombres (*shadowing*) ni asignación en la firma, se omite el uso de `this`. dentro de los métodos del objeto para mantener el código limpio.

Listas de Inicialización (*Initializer Lists*)

Se ejecutan **antes** de que el cuerpo del constructor corra y antes de que la superclase sea inicializada. Se definen mediante dos puntos `:` después de la firma del constructor y resultan esenciales para:

- Validar argumentos mediante sentencias `assert`.
- Inicializar campos `final` a partir de expresiones o valores calculados.
- Delegar la inicialización a otros constructores.

```
class RedServidor {
  final String url;
  final bool esSegura;

  // Lista de inicialización (ejecutada antes del cuerpo)
  RedServidor(String host, int puerto)
    : url = 'https://$host:$puerto',
      esSegura = puerto == 443 {
    print('Servidor configurado hacia $url');
  }
}
```

9.2 Patrón de Redirección y Constructores Factoría (*factory*)

Constructores Redirigidos (*Redirecting Constructors*)

Un constructor puede redirigir la responsabilidad de instanciación a otro constructor de la misma clase utilizando `this` en la lista de inicialización:

```
class Usuario {
  final String id;
  final String rol;
```

```
// Constructor principal
Usuario(this.id, this.rol);

// Constructor redirigido
Usuario.invitado() : this('INV-000', 'GUEST');
}
```

Constructores Factoría (factory)

A diferencia de los constructores generativos tradicionales, un constructor **factory**:

- **No crea automáticamente una nueva instancia** de la clase.
- Puede retornar una instancia **existente** desde una caché en memoria o retornar una subclase específica según los parámetros de entrada.
- Resulta ideal para implementar el patrón *Singleton* o manejar estrategias de *deserialización* desacopladas.

```
class Logger {
  final String nombre;
  static final Map<String, Logger> _cache = {};

  // Constructor factory que implementa un caché
  factory Logger(String nombre) {
    return _cache.putIfAbsent(nombre, () => Logger._interno(nombre));
  }

  // Constructor nombrado privado
  Logger._interno(this.nombre);
}
```

9.3 La Anotación @override y Contratos Metódicos

La anotación **@override** indica explícitamente al compilador que un método, getter, setter o atributo está redefiniendo un miembro heredado de una superclase o interfaz.

Aunque en Dart omitir la anotación **@override** no genera un error de compilación si la firma coincide, su inclusión es una **buena práctica obligatoria** para:

- Prevenir errores tipográficos en el nombre de los métodos heredados.
- Garantizar la intencionalidad del diseño ante refactorizaciones de software.
- Sobrescribir métodos clave de **Object** como **toString()**, **operator ==** y **hashCode**.

9.4 Ejemplo

Creen el archivo `bin/d9_constructores_override.dart` en VS Code. El siguiente programa demuestra el uso de listas de inicialización, constructores redirigidos, factorías y la sobrescritura del método `toString()` usando `@override`.

```
/// Módulo D9: Constructores Avanzados, this y @override en Dart
/// Demostración de_INITIALIZER Lists, Factory Constructors y Sobrescritura de
Métodos.

void main() {
  print('=====');
  print('    GESTIÓN DE SESIONES Y USUARIOS (D9 POO) - ESCOM    ');
  print('=====\\n');

  // 1. Uso de constructor redirigido (.invitado)
  print('--- 1. Instanciación con Constructor Redirigido ---');
  final usuarioInvitado = SesionUsuario.invitado();
  print('Perfil creado: $usuarioInvitado\\n'); // Invoca automáticamente toString()

  // 2. Uso de_INITIALIZER List con validación assert y campos calculados
  print('--- 2. Instanciación con Listas de Inicialización ---');
  final usuarioAdmin = SesionUsuario(
    id: 'USR-2026',
    email: 'OZZY@ESCOM.IPN.MX',
    nivelAcceso: 3,
  );
  print('Perfil creado: $usuarioAdmin\\n');

  // 3. Uso de Factory Constructor para resolución dinámica de tipos
  print('--- 3. Factory Constructor desde Payload JSON ---');
  final Map<String, dynamic> jsonAdmin = {
    'id': 'ADM-901',
    'email': 'admin@ipn.mx',
    'nivel': 5,
  };

  final Map<String, dynamic> jsonAnonimo = {
    'id': 'GUEST-11',
    'email': '',
    'nivel': 0,
  };

  final sesion1 = SesionUsuario.desdeJson(jsonAdmin);
  final sesion2 = SesionUsuario.desdeJson(jsonAnonimo);

  print('Sesión 1 (Factory): $sesion1');
  print('Sesión 2 (Factory): $sesion2');

  print('\\n=====');
}

/// Clase de entidad que representa la sesión de un usuario.
class SesionUsuario {
```

```
final String id;
final String emailNormalizado;
final int nivelAcceso;
final bool esSuperUsuario;

// Constructor Principal con Listas de Inicialización
SesionUsuario({
  required this.id,
  required String email,
  required this.nivelAcceso,
}) : assert(nivelAcceso >= 0 && nivelAcceso <= 5, 'El nivel de acceso debe
estar entre 0 y 5.'),
    emailNormalizado = email.trim().toLowerCase(),
    esSuperUsuario = nivelAcceso >= 3;

// Constructor Redirigido (.invitado)
SesionUsuario.invitado()
  : this(
    id: 'GUEST-000',
    email: 'guest@escom.ipn.mx',
    nivelAcceso: 0,
  );

// Constructor Factory para control flexible de instanciación
factory SesionUsuario.desdeJson(Map<String, dynamic> json) {
  final int nivel = json['nivel'] as int? ?? 0;

  if (nivel <= 0) {
    // Redirige a la instancia predeterminada de invitado si carece de nivel
    return SesionUsuario.invitado();
  }

  return SesionUsuario(
    id: json['id'] as String,
    email: json['email'] as String,
    nivelAcceso: nivel,
  );
}

// Sobrescritura explícita del método heredado de Object
@override
String toString() {
  return 'SesionUsuario[ID: $id | Email: $emailNormalizado | Nivel: $nivelAcceso
| SuperUser: $esSuperUsuario]';
}
}
```

Instrucciones de Ejecución en VS Code:

```
dart run bin/d9_constructores_override.dart
```

9.5 Ejercicios Prácticos

Implementen los siguientes dos programas en su entorno local para afianzar el uso de listas de inicialización, factorías y sobrescritura de métodos.

Ejercicio 1: Inicializador de Conexión a Base de Datos Local (Isar/SQLite)

Crea una clase llamada `ConexionBD`:

1. Atributos: `puerto` (`int`), `baseDatos` (`String`), `cadenaConexion` (`String`) y `esModoSeguro` (`bool`). Todos deben ser campos `final`.
2. Implementa un constructor principal que reciba `puerto` y `baseDatos`, y utilice una **lista de inicialización** para:
 - Construir `cadenaConexion` con la sintaxis: `'bd://localhost:$puerto/$baseDatos'`.
 - Asignar `esModoSeguro = true` únicamente si el `puerto` es igual a 5432 o 3306.
3. Sobrescribe el método `@override String toString()` para devolver una representación legible de la configuración de la base de datos.
4. Instancia la clase en `main()` para un puerto seguro y uno no seguro y muestra los resultados.

Ejercicio 2: Factory Cache para Gestor de Claves de Configuración

Escribe una clase `ConfiguracionApp`:

1. Atributos: `clave` (`String`) y `valor` (`String`), ambos `final`.
 2. Define un atributo estático y privado `static final Map<String, ConfiguracionApp> _instancias = {};` para gestionar la memoria caché.
 3. Implementa un **constructor nombrado privado** `ConfiguracionApp._crear(this.clave, this.valor)`.
 4. Define un **constructor factory** `ConfiguracionApp({required String clave, required String valor})` que busque si la `clave` ya existe en el mapa `_instancias`:
 - Si la clave ya está registrada, retorna la instancia previamente almacenada en el mapa (ignorando el nuevo valor).
 - Si no existe, crea una nueva instancia mediante el constructor privado, la almacena en `_instancias` y la retorna.
 5. Sobrescribe `@override String toString()`.
 6. En `main()`, intenta crear múltiples configuraciones con la misma clave para comprobar que el objeto se reutiliza sin duplicar memoria.
-